

App ExRE

Proyecto practico: Explorador de recursos de estudio

Asignatura: Programación para Dispositivos Moviles

Tecnologia base: Flutter para Android

Confeccionado por: Ing. Roilán Rodríguez Castillo

Entrega: Miercoles 30/09/2026 antes de las 23:59

Contexto

En este proyecto, debes construir una aplicacion Flutter llamada **ExRE** (Explorador de Recursos de Estudio). La app permitira consultar recursos de aprendizaje, filtrarlos, ver detalles, marcar favoritos, revisar progreso y navegar entre seis pantallas. La aplicacion no requiere servidor, base de datos remota ni autenticacion. Los datos deben provenir de un archivo local de prueba dentro del proyecto.

Alcance del proyecto

La app debe funcionar en Android y estar construida en Flutter. Debe incluir seis pantallas, navegacion basica con BottomNavigation, datos locales, favoritos y progreso. No se aceptan manejadores de estados mas alla de los que vienen en el SDK de Flutter.

Requisitos funcionales

RF1. Pantalla Inicio

La pantalla de inicio debe mostrar:

- Titulo de la app.
- Resumen del proposito del Explorador de recursos de estudio.
- Accesos rapidos a Catalogo, Galeria, Favoritos y Progreso.
- Indicador del estado actual de ciclo de vida observado desde Flutter.
- Contador de recursos disponibles, favoritos y recursos completados.
- Mensaje diferente cuando la app este en `resumed`, `inactive`, `paused` o `detached`.

RF2. Pantalla Catalogo

La pantalla de catalogo debe mostrar:

- Lista vertical de recursos de estudio.
- Búsqueda por titulo, categoria o autor.

- Filtro por categoria.
- Uso obligatorio de `ListView.builder` para listas largas.
- Uso de `ListView.separated` en al menos una seccion o vista alternativa.
- Acceso a la pantalla Detalle al tocar un recurso.
- Boton para marcar o desmarcar favorito desde la tarjeta o el detalle.

RF3. Pantalla Galeria

La pantalla de galeria debe mostrar:

- Coleccion visual de recursos en formato de cuadrícula.
- Uso obligatorio de `GridView.builder`.
- Dos columnas como minimo.
- Espaciado uniforme entre tarjetas.
- Acceso a Detalle al tocar una tarjeta.
- Indicador visual para recursos favoritos o completados.

RF4. Pantalla Detalle

La pantalla de detalle debe mostrar:

- Titulo, categoria, autor, duracion estimada, nivel y descripcion.
- Contenido o enlace simulado del recurso.
- Boton para marcar o desmarcar favorito.
- Boton para marcar como completado o pendiente.
- Boton para volver al listado anterior.
- Layout adaptable a pantallas pequenas sin overflow.

RF5. Pantalla Favoritos

La pantalla de favoritos debe mostrar:

- Recursos marcados como favoritos.
- Lista construida con `ListView.builder` o `ListView.separated`.
- Mensaje de lista vacia cuando no haya favoritos.
- Acceso a Detalle.
- Posibilidad de quitar un favorito.

RF6. Pantalla Progreso

La pantalla de progreso debe mostrar:

- Porcentaje de recursos completados.
- Cantidad de recursos completados y pendientes.
- Resumen por categoria.
- Lista de recursos completados.
- Actualizacion inmediata despues de cambiar el estado de un recurso.
- Indicador del ultimo evento de ciclo de vida registrado.

RF7. Navegacion basica

La app debe incluir navegacion entre las seis pantallas:

- Inicio -> Catalogo.
- Inicio -> Galeria.
- Inicio -> Favoritos.
- Inicio -> Progreso.
- Catalogo -> Detalle.
- Galeria -> Detalle.
- Favoritos -> Detalle.
- Detalle -> pantalla anterior (Según corresponda).

Se puede usar `Navigator.push`, `Navigator.pushNamed` o `go_router`. La implementacion debe ser coherente en toda la app (Solo se puede usar 1, nunca ambos).

Estado y ciclo de vida

Modelo de estado recomendado

La app debe mantener al menos:

- Lista de recursos de estudio.
- Conjunto de identificadores favoritos.
- Conjunto de identificadores completados.
- Categoria seleccionada.
- Texto de busqueda.
- Identificador del recurso abierto en Detalle.
- Ultimo `AppLifecycleState` observado.
- Historial breve de eventos de ciclo de vida.

Requisitos de scroll

ListView

- Usar `ListView()` solo para pocos elementos conocidos.
- Usar `ListView.builder` para listas largas o de tamaño variable.
- Usar `ListView.separated` cuando haya divisores visuales entre elementos.
- Definir `itemCount` siempre que la cantidad de elementos sea conocida.
- Usar `itemExtent` si todos los elementos tienen la misma altura.

GridView

- Usar `GridView.builder` para la galería.
- Configurar `SliverGridDelegateWithFixedCrossAxisCount`.
- Definir `crossAxisCount`, `crossAxisSpacing` y `mainAxisSpacing`.
- Ajustar `childAspectRatio` para evitar celdas deformadas.

Datos de prueba

El proyecto debe incluir al menos 20 recursos de estudio. Cada recurso debe tener:

- Identificador único.
- Título.
- Categoría.
- Autor.
- Duración estimada en minutos.
- Nivel: básico, intermedio o avanzado.
- Descripción breve.
- Tipo: video, lectura, práctica o documento.
- Portada local o color representativo.
- Estado inicial: pendiente.

Categorías sugeridas:

- Flutter.
- Android.
- Layouts.
- Scrollables.
- Slivers.
- Navegación.

Entregables

Cada estudiante debe entregar:

1. Proyecto Flutter completo y compilable.

2. Capturas de pantalla de las seis pantallas.
3. Breve README con instrucciones de ejecucion.

Criterios de aceptacion

- La app inicia sin errores en Android.
- No hay overflow visible en las pantallas principales.
- El catalogo muestra al menos 20 recursos.
- La galeria usa `GridView.builder` (mínimo 2 columnas).
- El detalle abre desde catálogo, galeria y favoritos.
- Los favoritos se actualizan en todas las pantallas.
- El progreso cambia despues de marcar un recurso como completado.
- El codigo es legible y mantiene separacion entre datos, estado, pantallas y widgets reutilizables.